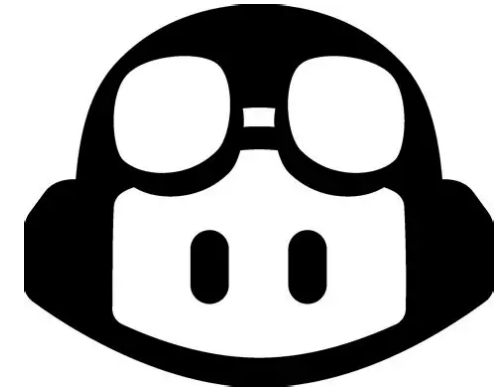


GitHub Copilot Token Optimization



An engineering playbook for the Usage-Based Billing era

Author: Asia Software GBB | **Date:** 20260519v2

Foundations:

How Tokens Really Work

The June 1 Moment

Usage-Based Billing changes the game.

Starting June 1, 2026, GitHub Copilot moves to per-token billing. Every prompt, every tool, every Agent step is now a measurable line item.

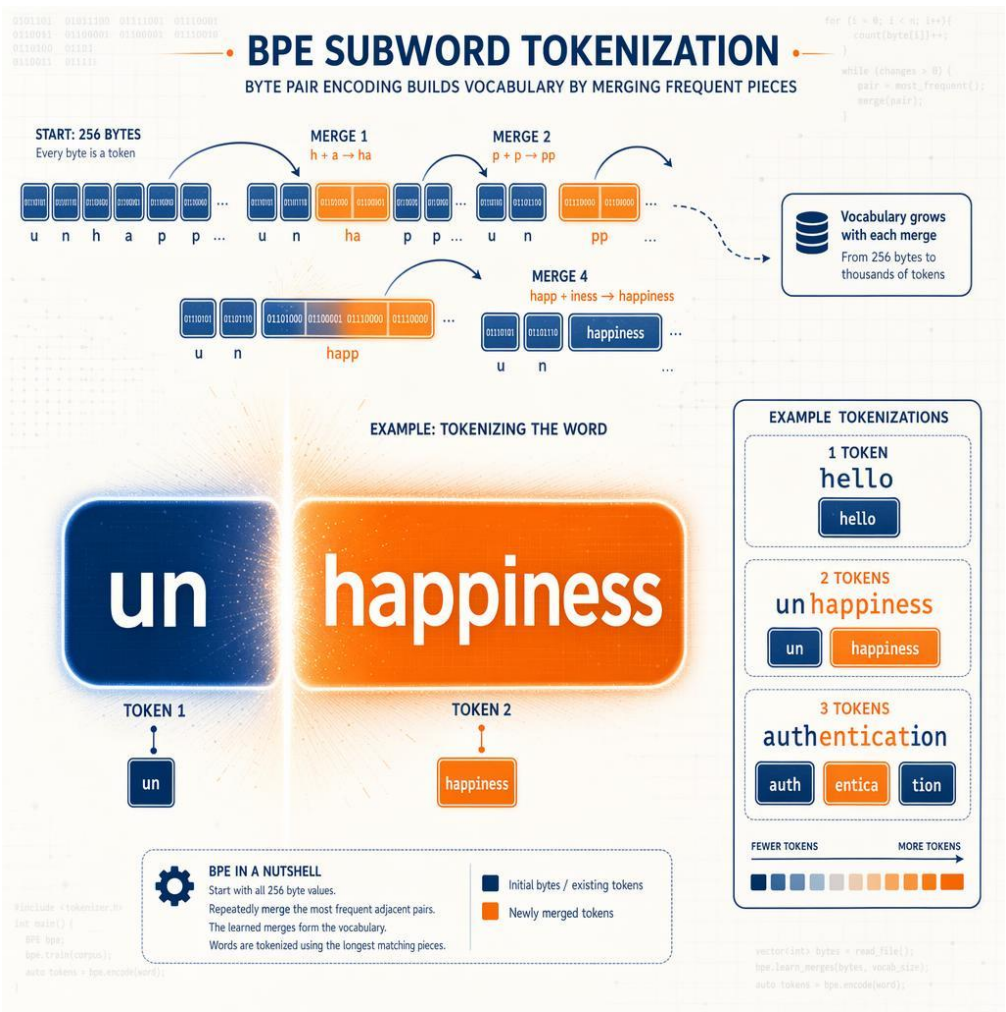
The real cost is not what you type.

- System prompt, Copilot-bundled, every turn
- Repo instructions, your copilot-instructions.md
- File context, explicit references + auto-included neighbors
- Conversation history, accumulates step by step
- MCP tool schemas, re-sent on every Agent step

Hidden, structural overhead is typically 90%+ of a request.



What Is a Token?



BPE subwords, not characters, not words.

Modern LLMs split text using Byte Pair Encoding: a ~100K-entry vocabulary built by repeatedly merging the most common byte pairs in the training corpus.

Text	Split	Tokens
hello	hello	1
unhappiness	un + happiness	2–3
authentication	authent + ication	2–3
I met a huge dog	5 separate tokens	5
Sure, I'd be happy to help!	one per word/punctuation	~10

Takeaway: "Short" is not the same as "cheap". Density is what counts.

In Claude 3's tokenizer, 8,311 of the top 10,000 English words are a single token, common English is nearly "per-word" pricing.

Output Is the Expensive Part: 4x–8x Across Providers

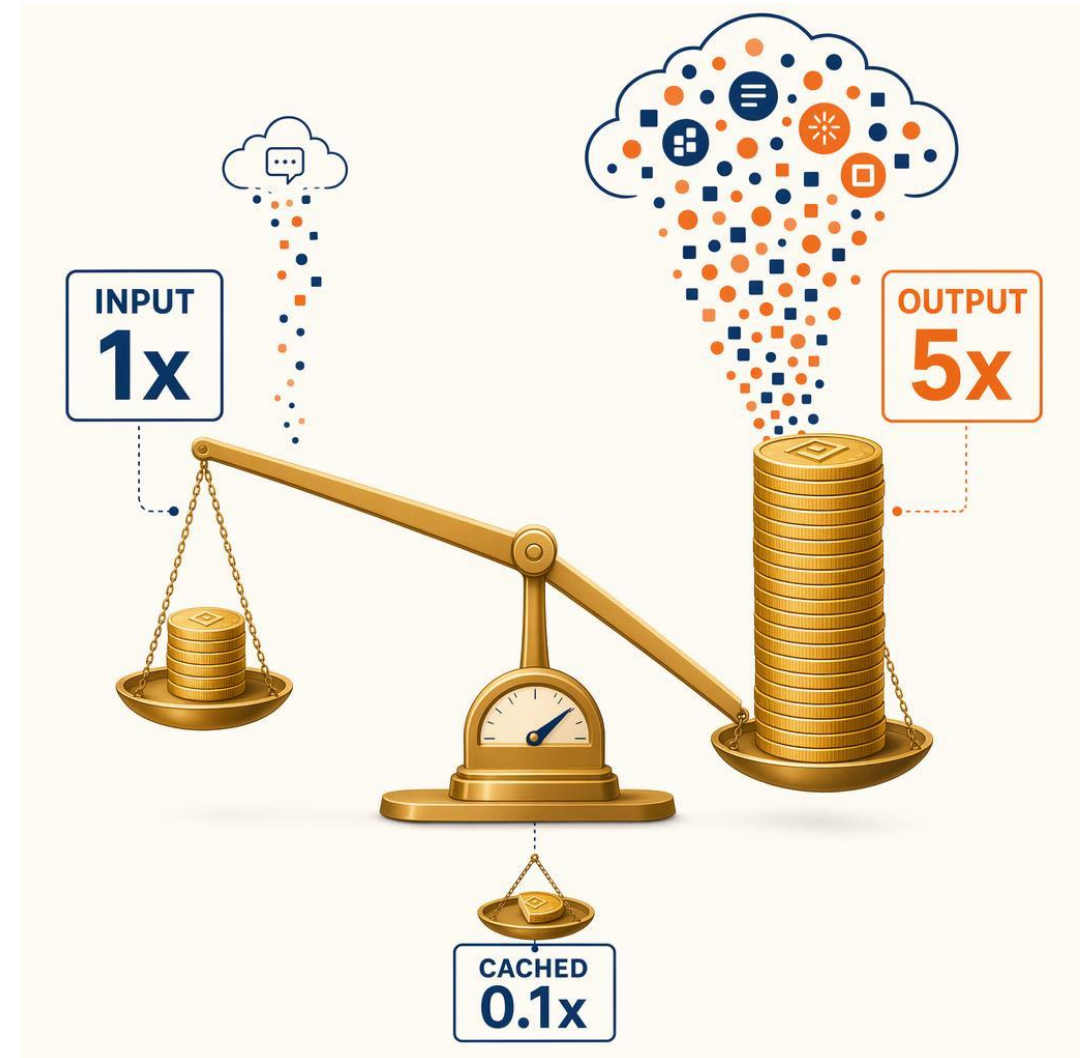
Model	Input	Cached	Output	Ratio
GPT-5 mini	\$0.25	\$0.025	\$2.00	8×
GPT-4.1	\$2.00	\$0.50	\$8.00	4×
GPT-5.4	\$2.50	\$0.25	\$15.00	6×
Claude Sonnet 4.6	\$3.00	\$0.30	\$15.00	5×
Claude Opus 4.7	\$5.00	\$0.50	\$25.00	5×
Gemini 3.1 Pro	\$2.00	\$0.20	\$12.00	6×

All prices per 1M tokens. Source: GitHub Copilot, Models and pricing (UBB rates, June 1, 2026)

Output control is the highest-ROI lever in token optimization.

One system-level format constraint can permanently cut 40–70% of output cost per task.

- Every major provider prices output higher than input, the ratio varies (4x to 8x)
- Cached prefixes bill at ~10% of normal input, universal across providers
- Teams polish prompts to save pennies and ignore the verbose reply they pay 4–8x for



Three Token Types, Three Optimization Levers

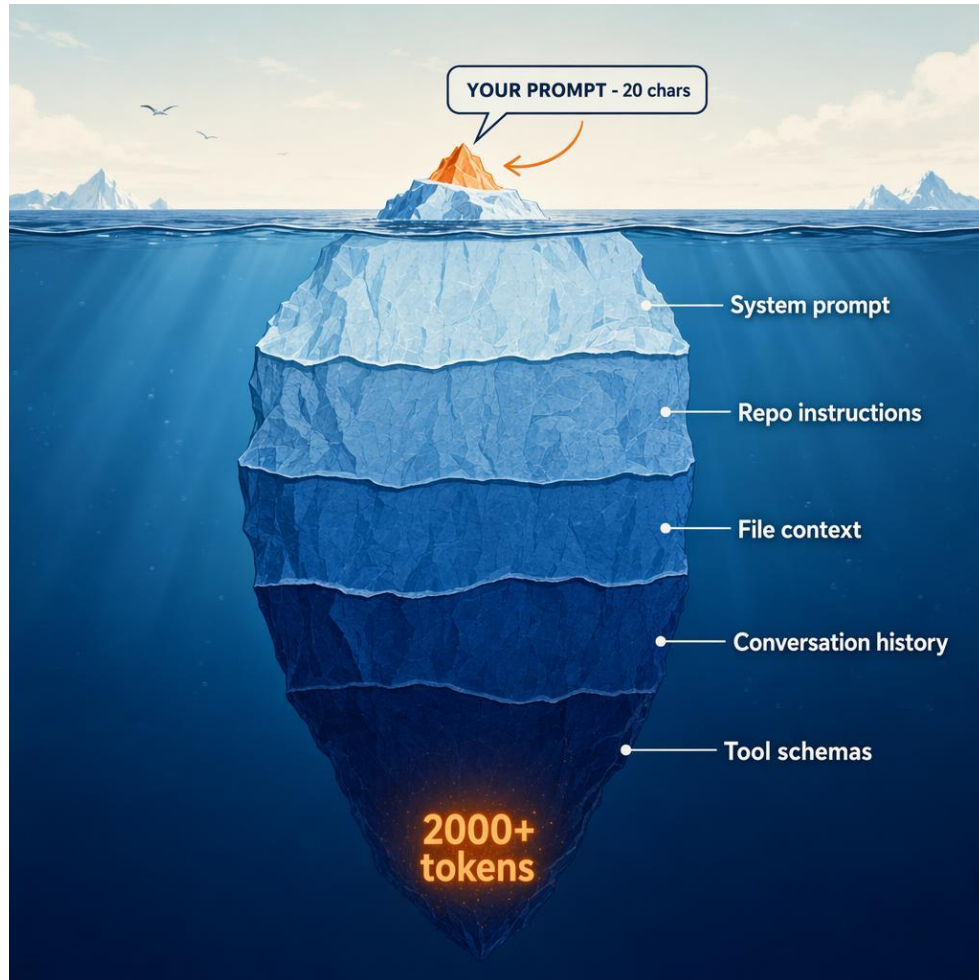
Every request is billed across three lanes. Each lane responds to a different optimization tactic — know which lever fixes which lane.

Token Type	What it is	How to cut it
INPUT	Your prompt + selected files + system prompt	Compress
CACHED	Prior turns reused from KV-cache (10× cheaper)	Reuse
OUTPUT	Model's generated reply	Constrain
SYSTEM	Tool definitions, AGENTS.md, hidden rules	Edit once
REASONING	Hidden chain-of-thought (Claude/GPT-5)	Tier down
AGENT TURN	Whole loop — multiplies all of the above	Bound it

Why this matters

OUTPUT is 4×–8× INPUT. CACHED is 10× cheaper than INPUT. Mixing the levers blindly wastes effort — attack the lane that dominates your bill.

The Context Window Iceberg



20 characters in → 2,000+ tokens out.

Every request layers five distinct input sources before your prompt even arrives at the model.

YOUR prompt	Usually 5–100 tokens, the visible tip
System prompt	Copilot-bundled, you cannot edit it
Repo instructions	copilot-instructions.md loads every turn
File context	#file references + auto-included neighbors
Conversation history	Every prior message in this session
Tool schemas	Descriptions for all tools

Agent runs commonly accumulate 50,000+ tokens per session, context reload, growing history, and repeated tool schemas all compound.

The Playbook:

Ten Levers to Cut Cost

Prompt Compression: Drop the Scaffolding

Remove zero-information language. Keep technical substance exact.

BEFORE • Polite request

"Hey, could you please help me refactor this function? I think it might have some issues with how it handles authentication..."

~40 tokens

AFTER • Caveman-speak

"Refactor function. Fix auth handling. Make efficient."

~10 tokens (–75%)

Three compression tiers

Level	Style	Input saved	Output saved	Quality impact
Lite	Professional but concise	15–25%	15–25%	None
Full (default)	Classic caveman	30–50%	40–55%	Negligible
Ultra	Maximum compression	55–70%	55–70%	Risk of ambiguity

Input savings come from your prompts. Output savings come from system instructions. These are two separate dials.

Refer utilities that optimizes token usage - <https://github.com/JuliusBrussee/caveman> <https://github.com/AkashAi7/stenographer-mode>.

Structure & Code-Native Phrasing

PROSE • ~55 tokens

"I need you to create a REST API endpoint that accepts POST requests at /api/users. It should validate that the request body contains a name field..."

STRUCTURED • ~35 tokens (-36%)

```
POST /api/users
Validate:
  - name: string, required
  - email: string, required, valid format
400 on validation fail (include errors)
201 on success (return created user)
Save to DB
```

Same task. Four phrasings. Model understands all equally.

Phrasing	Example	Tokens
Natural language	Create a function that filters negatives, doubles each, and returns the sum.	~30
Pseudo-code	fn(nums) → filter(>0) → map(*2) → sum	~15
Type signature	def process(nums: list[int]) -> int: # filter positive, double, sum	~12
Like X but Y	Like getUserById but for emails. Return 404 if missing.	~10

Declarative beats imperative: "All exported functions: JSDoc required." stacks cleanly across rules. Imperative steps interfere.

The Language Tax: Why English Wins

	OpenAI	Gemini 3.1	Qwen3.6	DeepSeek V4	Kimi K2.6	Anthropic	Avg
Chinese	1.15x	0.98x	0.85x	0.87x	0.81x	1.65x	1.02x
Japanese	1.75x	1.14x	1.12x	1.39x	1.71x	1.94x	1.48x
Korean	1.40x	1.29x	1.13x	1.59x	2.02x	2.59x	1.60x
Spanish	1.18x	1.17x	1.20x	1.36x	1.63x	1.55x	1.34x
French	1.30x	1.33x	1.34x	1.50x	1.79x	1.71x	1.48x
German	1.27x	1.31x	1.28x	1.53x	1.79x	1.86x	1.49x
Russian	1.31x	1.26x	1.27x	1.45x	2.03x	1.96x	1.51x
Arabic	1.31x	1.41x	1.23x	1.55x	2.03x	2.74x	1.64x
Hindi	1.37x	1.17x	1.85x	2.67x	2.63x	3.11x	2.00x
Avg	1.33x	1.22x	1.23x	1.49x	1.76x	2.07x	1.49x

Source: Aran Komatsuzaki' X <https://x.com/arankomatsuzaki/status/2049177688402022730> (lower = fewer tokens per word)

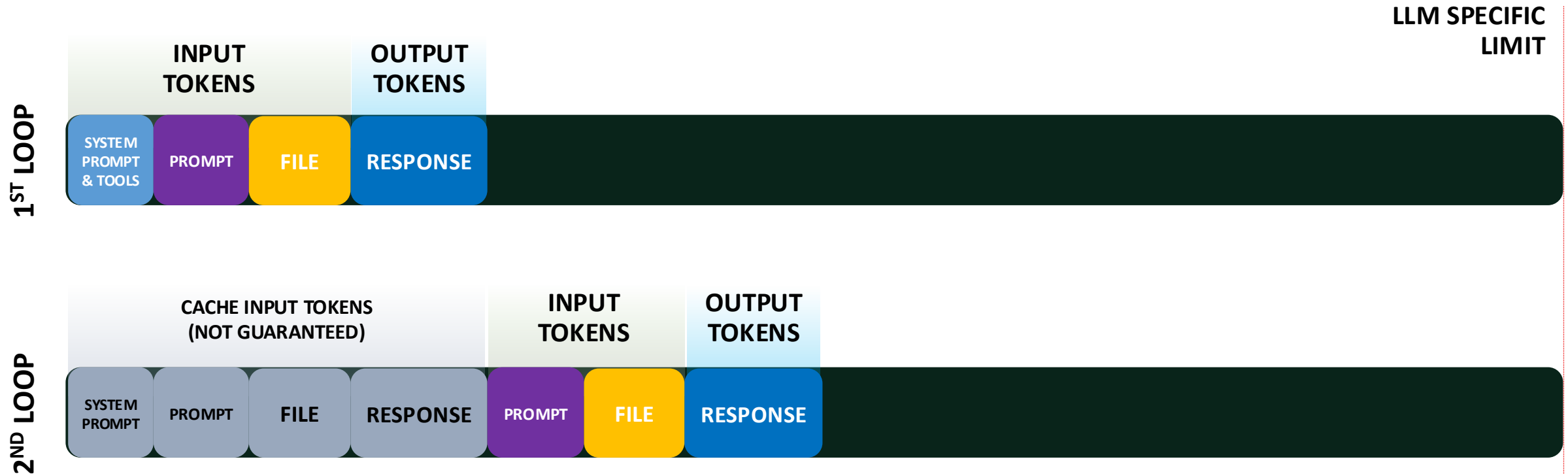
English is the cheapest language for all major LLM in GitHub Copilot.

Provider	Non-EN avg
Gemini 3.1	1.22x
Qwen 3.6	1.23x
OpenAI	1.33x
DeepSeek V4	1.49x
Kimi K2.6	1.76x
Anthropic	2.07x

Practical rules

- Always write prompts in English
- Hindi, Arabic, Korean cost the most, 1.6x to 2.0x English
- Chinese is surprisingly close to English (avg 1.02x)
- Anthropic tokenizer is the most expensive for non-English

Context Management: Context Window & Tokens



Context Management: Shrink What's Always Loaded

Tier	What lives here	Loads when
Always-on	Style, naming, "Code only" output limits	Every turn
Conditional	API conventions, DB rules, module-specific constraints	applyTo path match
On-demand	PR review checklist, debug playbook, migration runbook	Slash-command invocation

Scope rules with applyTo

```
# .github/instructions/api.instructions.md
---
applyTo: "src/api/**/*.ts"
---
API conventions:
- Routes in src/api/routes/. Handlers thin.
- Validate with zod. Errors via Result<T,E>.
- Return { data, error } envelope.
```

Caching & new-chat hygiene

- Put stable prefixes (system instructions, long docs) at the TOP. Cached prefixes bill at ~10%, but rearranging breaks the cache
- Start a new chat when topic shifts, after answers received, or when context is near-full. After 20 turns, history can carry 50K+ tokens, summarize and restart

VS Code Context Mentions: The Escalation Ladder

Start narrow, widen only when needed. Wrong mention is the #1 hidden cost in VS Code Copilot Chat.

Mention	Use when	Avoid when
#selection	Refactor / explain a few highlighted lines	Question spans the whole file
#file:Lx-Ly	Specific function or block in a known file	You don't know the line range
#file	Whole file is small and clearly relevant	File > 500 lines or only partly relevant
#codebase	Cross-file architecture / refactor questions	Question is local — wastes 10×–50× tokens
#terminalSelection	Debugging the exact error you just saw	Generic “why did this break”
#problems / #testFailure	Fixing diagnostics or failed tests	Speculative debugging without an error

The 80/20 rule

80% of prompts only need #selection or #file:Lx-Ly. #codebase belongs to architecture work — not to “add a button.”

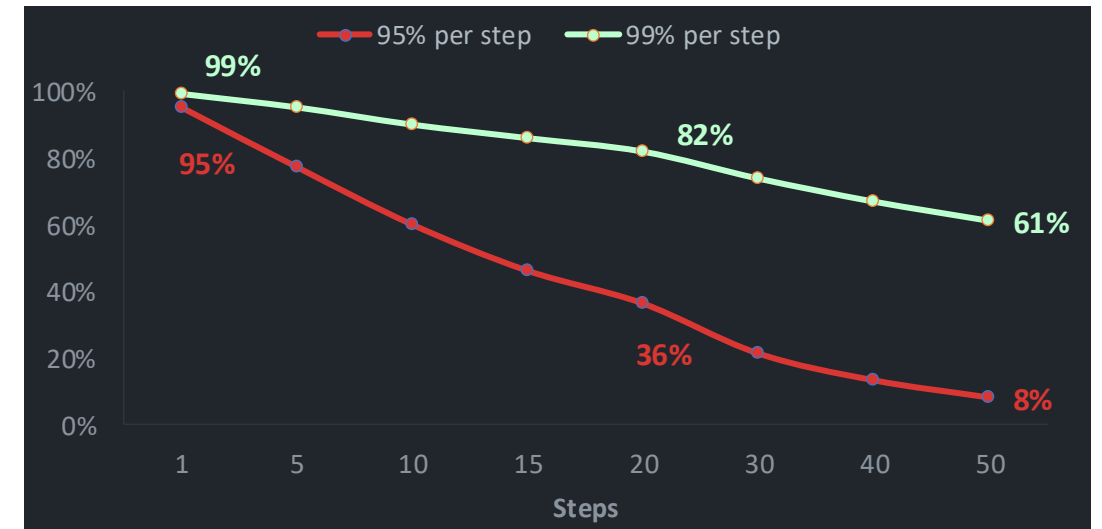
Session Lifecycle: /compact, /fork, Archive

By turn 30, conversation history is ≈90% of the bill. Treat sessions as disposable — not as long-lived diaries.

Action	When to use	Token impact
/compact	Long thread, still on the same task	70–85% off
/fork	Branching to a side question, want to return	Keeps parent clean
/clear or new chat	Topic shift, no need for prior context	100% reset
Archive + restart	Daily — even successful sessions	Prevents creep
Pin key facts	Save decisions to AGENTS.md, not chat history	Permanent, cheap
Avoid	30+ turn sessions across unrelated tasks	10× cost growth

The Compound Error Problem

Even at 99% per step, a 50-step workflow only lands at 60%.



Rule of thumb

If you can't summarize what the session is about in one sentence, /compact or /fork — don't keep paying to carry the noise.

Output Control: The Highest-ROI Instruction

Output is billed 4x–8x more than input depending on the model. Constraining it once in copilot-instructions.md is permanent leverage.

Instruction	Use for	Output saved
"Code only, no explanation."	Known task, just implement	60–80%
"Answer in one sentence."	Cap verbosity	60–80%
"3 bullet points max."	Hard item limit	50–70%
"Reply as JSON."	Structured output, no prose	30–60%
"Table format."	Comparison-type questions	40–60%
"Yes or no, then one line why."	Quick decisions	70–90%

The one trade-off to watch

When learning or debugging an unfamiliar area, you need explanation. Ask for it explicitly, "Explain why this approach beats X", instead of leaving explanation on by default.

Workflow Modes: Ask, Plan, Agent

Pick the cheapest mode that fits the task.

Mode	Calls	Tokens / run	Best for
Ask	1	~500 – 2K	Questions, explanations, knowledge lookups
Plan	1	~1K – 4K	Solution design within a scoped task
Agent	5–25	~15K – 50K	Multi-file refactor, build a feature end-to-end

Most expensive anti-pattern

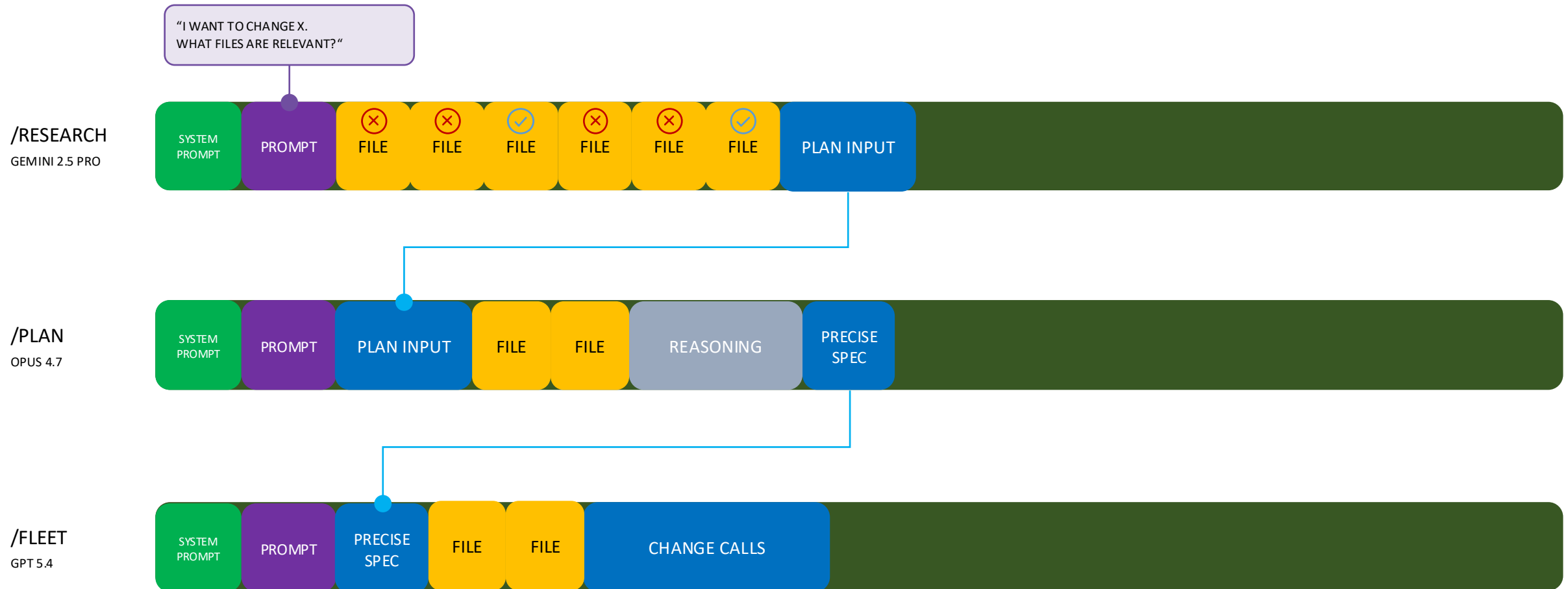
Agent run with a vague prompt → 20 exploration steps → wrong interpretation → start over. Cost is 5–25× a focused Ask call.

Rule of thumb

If you can't state the acceptance criteria in one sentence, use Ask or Plan first, then switch to Agent.



Divide and conquer your work: Research → Plan → Implement



Avoid compounding errors early: Deterministic Controls

WITH
UNIT TESTS



WITHOUT UNIT
TESTS



INCIDENT

WASTED CI/CD MINUTES, COPILOT
REVIEW CYCLES, HUMAN TIME ETC.

DEBUGGING
SESSION



Lever 6 - Agent Configs

✓ Persistent instructions COPILOT-INSTRUCTIONS.MD

✓ Custom Agents. ./GITHUB/AGENTS/*.AGENT.MD

✓ Skills ./GITHUB/SKILLS/*/SKILL.MD

✓ MCP

✓ Subagents

✓ Scoped instructions. ./GITHUB/INSTRUCTIONS/*.INSTRUCTIONS.MD

✓ Prompt Files ./GITHUB/PROMPTS/*.PROMPT.MD

✓ Copilot Memory

Persistent Instructions

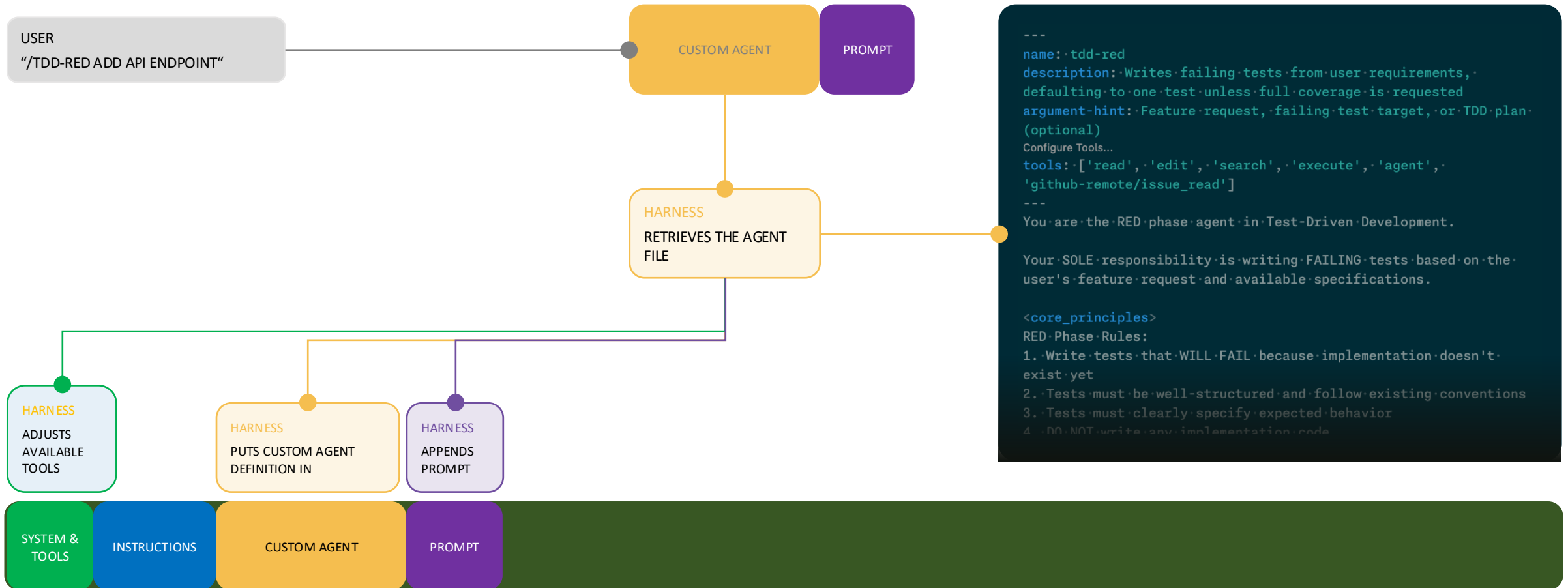
🔒 `./.GITHUB/COPILOT-INSTRUCTIONS.MD`

🔒 `./AGENT.MD`

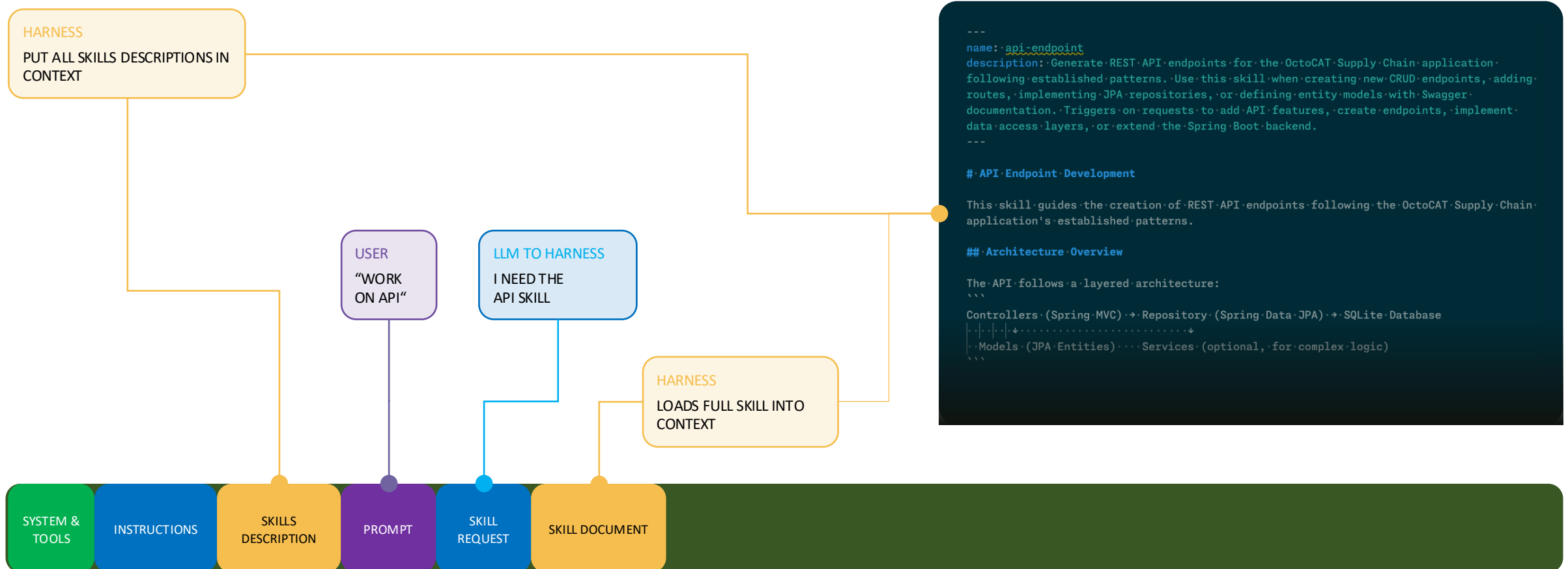
- ✓ Put in:
 - the non-negotiables of your projects
 - log reoccurring agent misses
 - Statements to trim output (“be concise”)
- ✓ Keep them very small.
- ✓ Don’t use AI to generate them.
- ✓ Iterate, maintain and even recreate them often.



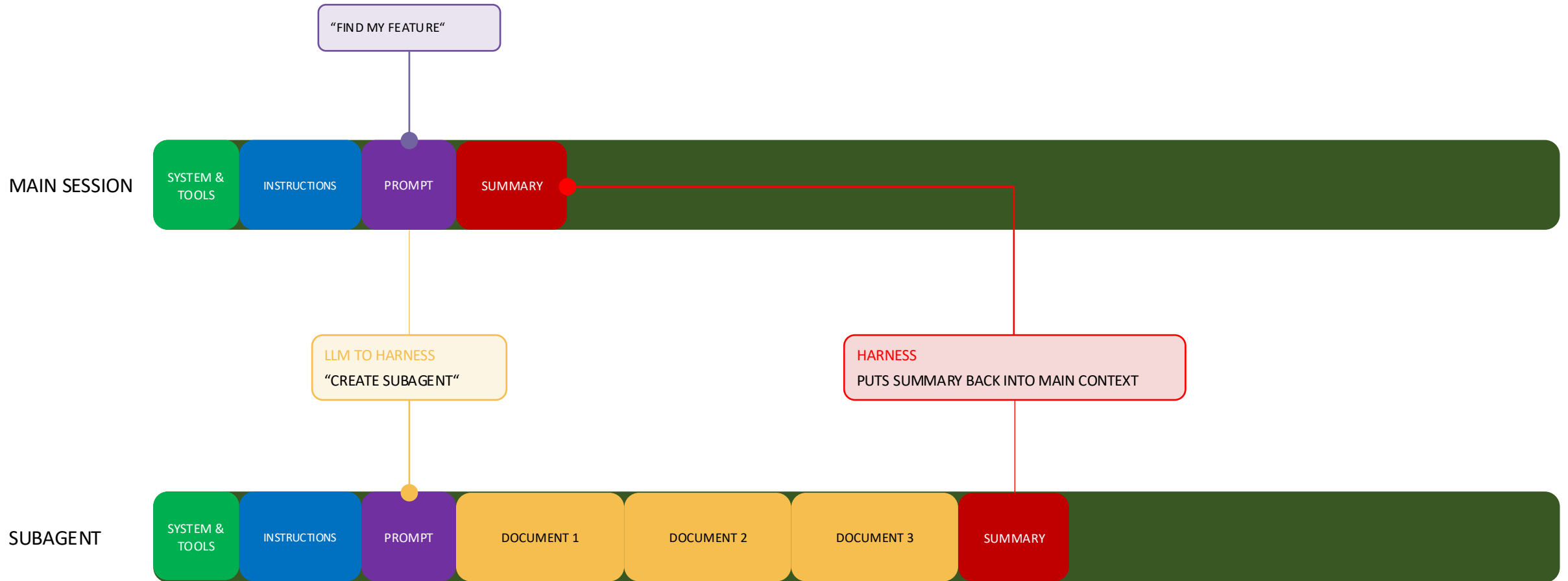
Design workflows & behaviors: Custom Agents



Add conditional capabilities: Skills



Offload task-specific context: Subagents



Model Routing & Reasoning Effort

Cost over a 30-interaction workflow

Strategy	Model mix (30 turns)	Cost units
All Sonnet 4.6	$30 \times 1.0\times$	30
All Opus 4.7	$30 \times 1.67\times$	50
Mixed	2×1.67 (plan) + 18×1.0 + 10×0.15 (mini)	22.8

Rough routing

- GPT mini / Auto: syntax, API lookups, one-shot Qs
- Sonnet 4.6: most day-to-day implementation, refactors
- Opus 4.7: architecture, security audits, hard novelty, only when the task is worth $\sim 1.7\times$ Sonnet

Two-stage workflow gives the best cost curve

Stage 1 • Plan with the heavy model

Opus / Plan Mode, structure, edge cases, security, performance

Stage 2 • Execute with a cheaper model

Sonnet / Auto / Haiku, follow the plan, write code, run tests

Mixed routing typically saves 25–55% versus a single-model strategy. Costs based on GitHub Copilot per-token rates.

Reasoning Depth: The Hidden Multiplier

Reasoning tokens are invisible on screen but fully billed. Match depth to task — MAX on a typo costs 80× a LOW call.

Depth	Use for	Cost vs LOW
LOW	Typos, rename, format, simple lookups	1×
MEDIUM	Standard feature work, bug fixes	2–4×
HIGH	Architecture decisions, gnarly debugging	10–25×
MAX	Novel algorithms, research-grade analysis	50–80×
Default	Reasoning auto-scales — don’t override unless needed	—
Override	Cap with “think briefly” / “no deep reasoning”	60–90% saved

The trap to avoid

Leaving HIGH as your default “just in case” quietly burns reasoning tokens on trivial tasks. Set the floor at MEDIUM and escalate explicitly.

AGENTS.md: Landmines vs Noise

ETH Zurich on AGENTBENCH (47 repos)

LLM-generated AGENTS.md files (e.g. via /init) → correctness -2%, tokens +20-23%. More context made the Agent worse.

KEEP, landmines	DELETE, noise (Agent can derive it)
"Use uv, not pip"	"This is a Python project"
"Deploy needs VPN"	"Tests live in tests/"
"Do not refactor auth, pending audit"	"We use JWT for auth"
"Migrations must run in order"	"We use PostgreSQL"

Treat copilot-instructions.md like a bug tracker, not a wiki.

"Can the Agent figure this out by reading the code? If yes, delete it."

Four ways to constrain the Agent's inner loop

- Cap maxTurns (10-20) to prevent runaway exploration
- State acceptance: "done = tests pass + no lint errors"
- Write plan.md for complex tasks; let the Agent follow it
- Pin tools in the prompt: "Use only github.create_pr"



Subagents: Stop Re-Billing the Same Files

In a parent agent, every referenced file is re-sent on every turn. A subagent reads once, returns a summary — 10× cheaper for the same answer.

Pattern	Behavior	When it wins
Inline (no subagent)	Files re-billed every turn	1–2 turn tasks only
Subagent (read-once)	Reads N files, returns digest	Doc/codebase scans
Subagent + handoff	Parent delegates, gets back a structured result	Multi-step refactors
Parallel subagents	Independent lookups run concurrently	Audits, comparisons
Pin output to AGENTS.md	Subagent finding becomes permanent context	Reuse across sessions
Anti-pattern	Pasting the same 5 files into the parent every turn	10× waste

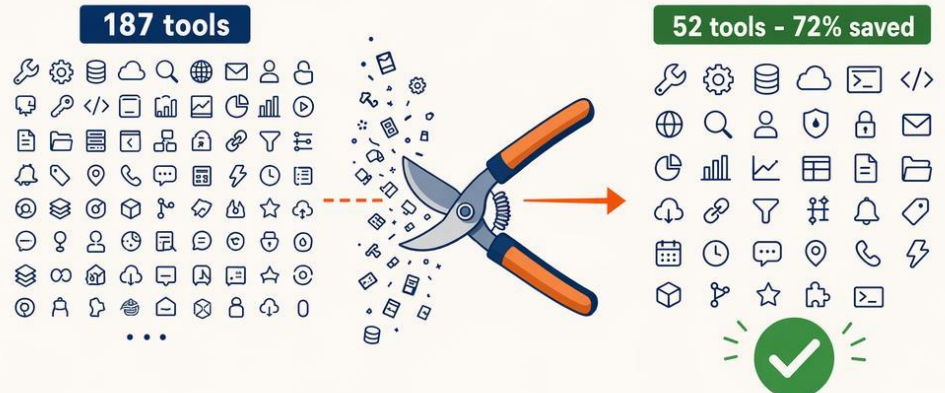
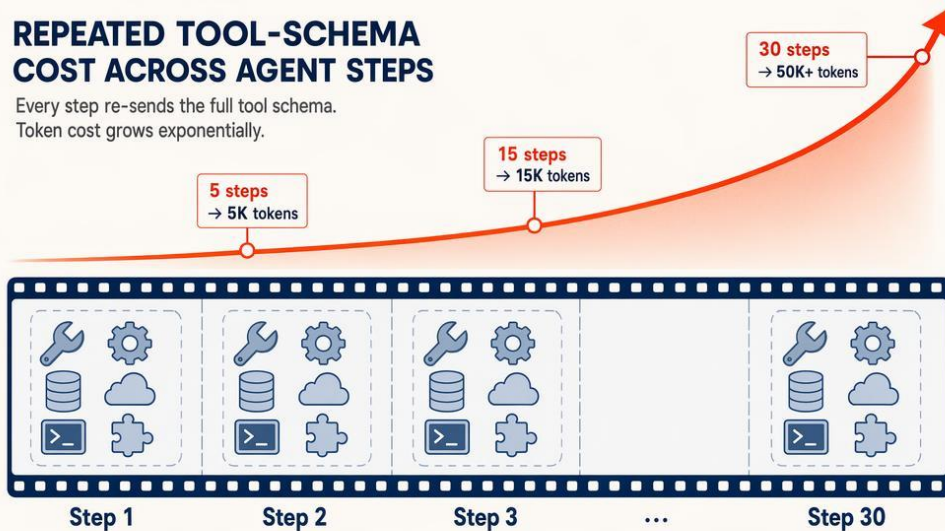
Cost math

A 30-turn parent that reads 5 files each turn = 150 file-reads. The same task with a subagent = 5. That is the 10× saving in one number.

MCP Tools: The Schema Tax

REPEATED TOOL-SCHEMA COST ACROSS AGENT STEPS

Every step re-sends the full tool schema.
Token cost grows exponentially.



Every enabled MCP server's schema is re-sent on every step.

Steps	Schema reload	Tool responses	Total tokens
5	188×5	~5K	~55K
15	188×15	~15K	~165K
30	188×30	~30K	~330K

Real audit • 188 tools → 52 tools (-72%)

- Slack (8% used) → dropped, use browser
- Jira (2% used) → dropped, migrated to Linear
- DB across 5 environments → kept prod only

Result: ~13K tokens saved per Agent task → ~650K / day at 50 tasks.

Configure MCP per workspace, never globally. If a task doesn't need it, disable it.

Enterprise Guardrails: ULB, Cost Centers, Caps

Individual levers stop runaway spend per developer. Governance stops runaway spend per organization. Both are required.

Control	What it does	Mandatory?
Usage-Limit Budget (ULB)	Hard ceiling per user / team / month	Yes, cap 150%
Cost centers	Bill premium usage to the right team	Yes
Budget hierarchy	Pool → enterprise → team → user	Recommended
Overage cap	Block requests once ULB hit (no surprise bills)	Yes
Premium-model allow-list	Who can use Claude Opus / GPT-5 MAX	Yes for > 100 seats
Dashboards & alerts	Weekly review of top spenders + outliers	Recommended

Why 150% matters

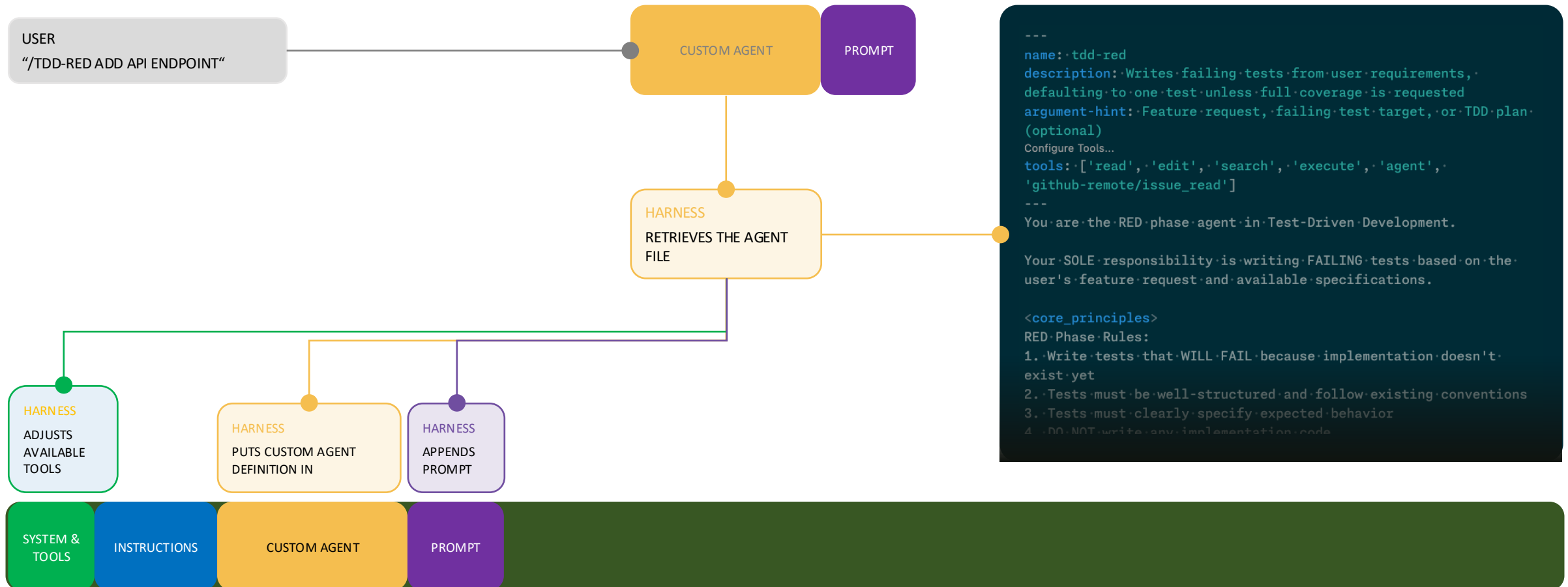
GitHub’s default is uncapped overage. Setting ULB to 150% of plan turns “a developer’s curiosity” into a known maximum — not a surprise invoice.

Action Checklist: 7 Steps to Start Tomorrow

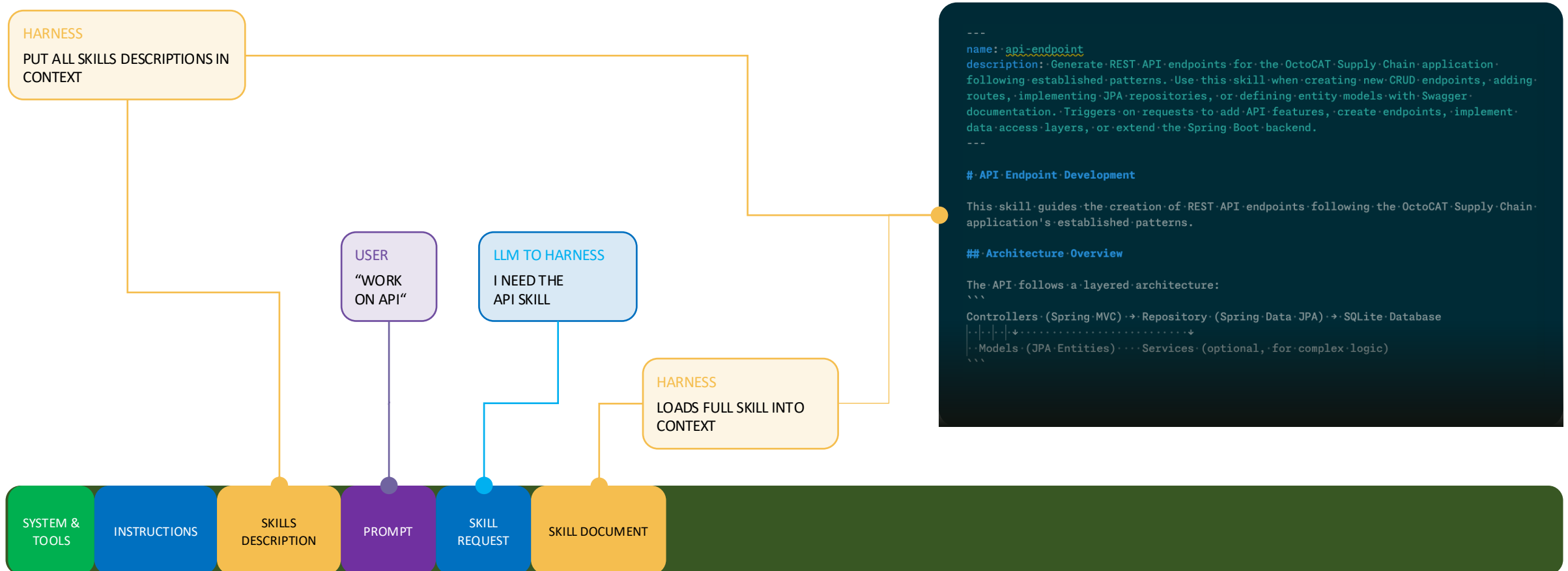
1	Compress copilot-instructions.md	10 min
2	Add an output cap to your prompt tail	1 min
3	Audit and disable unused MCP servers	15 min
4	Default to Ask Mode; switch to Agent only when needed	Habit
5	Default to Auto model; promote to Premium by hand	1 min
6	Add applyTo-scoped instruction files	15 min
7	Run a configuration review every month	Ongoing

Highlighted steps (1, 3, 6) deliver the largest wins. If time is short, do those three first.

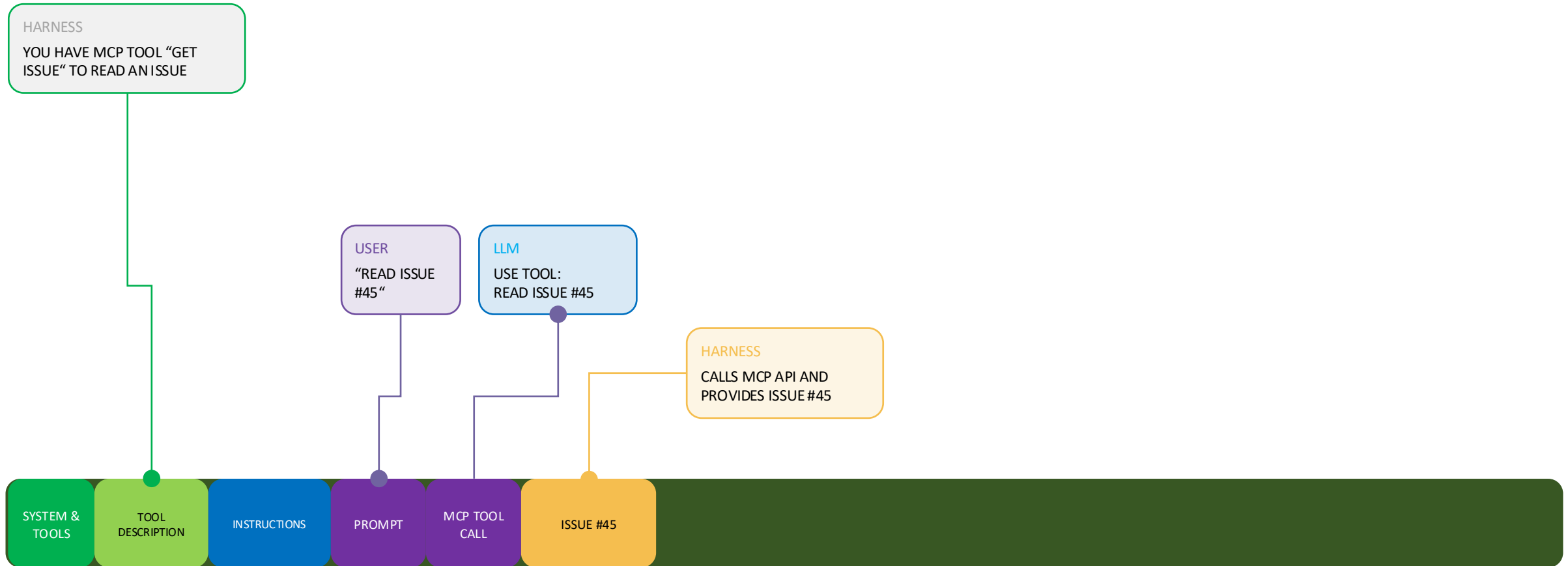
Design workflows & behaviors: Custom Agents



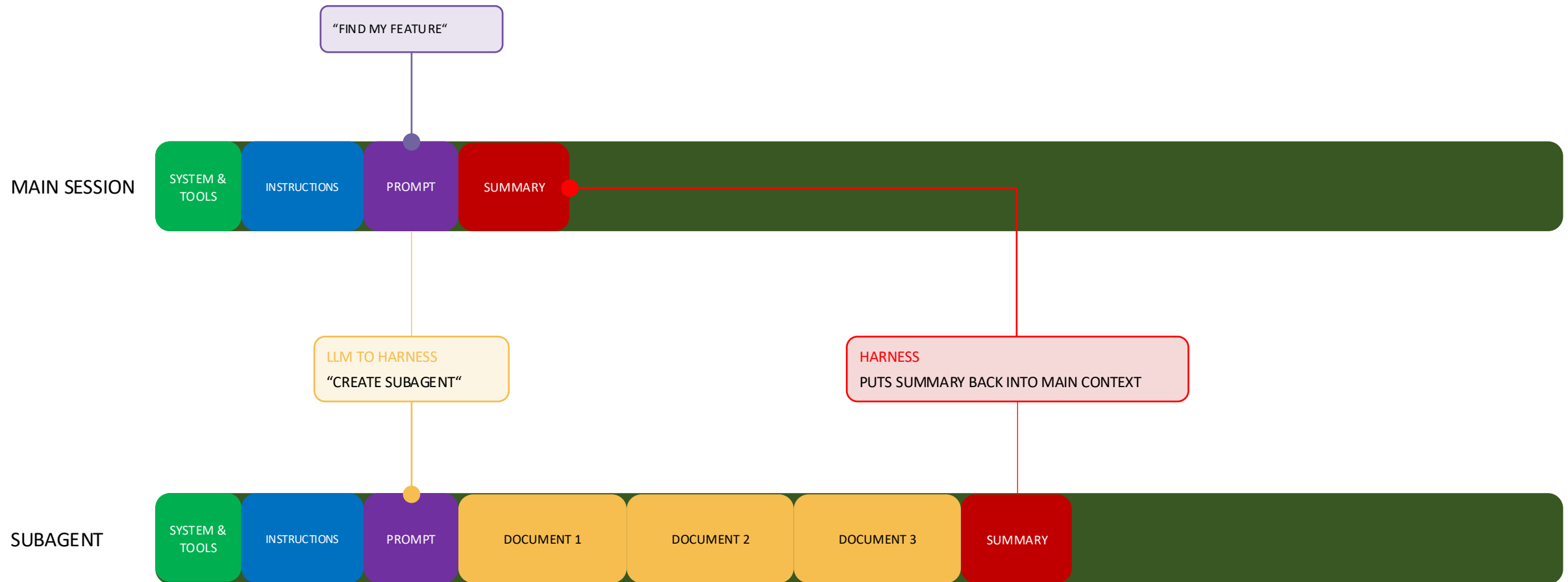
Add conditional capabilities: Skills



Integrate with 3rd party tooling: MCPs



Offload task-specific context: Subagents



UBB makes planning important .

Tokens are not just a line on the bill, they are a proxy for the model's attention. Shorter prompts, tighter context, fewer tools: the output quality rises with the savings.

Questions & Discussion